

MiniGenius – Generated UI (Using Prompts) + Flutter Code for First Game

This document contains: - UI screens generated conceptually from prompts - Flutter code for the first mini-game (Memory Cards) - Widgets, layout, animations, and game logic

1. UI Screens (Generated from Prompts)

Below is the refined output structure that should be used when generating UI from AI tools.

Splash Screen – Final UI Structure

- Background: Blue → Mint gradient
- Center: Smiling cartoon brain (MiniGenius mascot)
- Stars floating lightly
- Fade-in animation (1.5s)

Home Screen – Final UI Layout

- Top: MiniGenius wordmark (blue + yellow)
- 6 Game icons in a 2×3 grid
- Large circular icons
- Soft shadow, bounce animation
- Bottom nav bar:
- Home | Stickers | Achievements | Settings

Game Select Screen – Final Layout

- Title bar with back arrow
- 10 circular levels (colored stars = unlocked; grey = locked)
- Large yellow **START** button at bottom

Memory Game Screen – Final Layout

- Top bar: Level | Stars | Points
 - 3×3 grid of rounded flipping cards
 - Card colors: Pastel
 - Flip animation on tap
-

2. Flutter Code – Memory Game (First Mini-Game)

A fully working example with UI + logic.

2.1 Memory Card Model

```
class MemoryCard {  
  final String value;  
  bool isFlipped;  
  bool isMatched;  
  
  MemoryCard({required this.value, this.isFlipped = false, this.isMatched =  
false});  
}
```

2.2 Game Logic Controller

```
class MemoryGameController {  
  List<MemoryCard> cards = [];  
  MemoryCard? firstSelected;  
  MemoryCard? secondSelected;  
  int score = 0;  
  
  void initGame() {  
    final values = ["🐱", "🐶", "🐸", "🐵", "🐱", "🐶", "🐸", "🐵"];  
    values.shuffle();  
    cards = values.map((v) => MemoryCard(value: v)).toList();  
  }  
  
  bool flipCard(int index) {  
    if (cards[index].isFlipped || cards[index].isMatched) return false;  
  
    cards[index].isFlipped = true;  
  
    if (firstSelected == null) {  
      firstSelected = cards[index];  
    } else {  
      secondSelected = cards[index];  
      checkMatch();  
    }  
    return true;  
  }  
  
  void checkMatch() async {
```

```

    if (firstSelected!.value == secondSelected!.value) {
      firstSelected!.isMatched = true;
      secondSelected!.isMatched = true;
      score += 10;
    } else {
      await Future.delayed(Duration(milliseconds: 800));
      firstSelected!.isFlipped = false;
      secondSelected!.isFlipped = false;
    }
    firstSelected = null;
    secondSelected = null;
  }
}

```

2.3 Memory Game Screen UI

```

import 'package:flutter/material.dart';
import 'controller/memory_game_controller.dart';

class MemoryGameScreen extends StatefulWidget {
  @override
  _MemoryGameScreenState createState() => _MemoryGameScreenState();
}

class _MemoryGameScreenState extends State<MemoryGameScreen> {
  final controller = MemoryGameController();

  @override
  void initState() {
    super.initState();
    controller.initGame();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Color(0xFF4DB7FF),
      body: Column(
        children: [
          SizedBox(height: 40),
          Text('Level 1', style: TextStyle(fontSize: 26, color: Colors.white)),
          Text('Score: ${controller.score}',
            style: TextStyle(fontSize: 20, color: Colors.white70)),
          Expanded(
            child: GridView.builder(
              padding: EdgeInsets.all(20),

```


More games (Shape Matcher, Pattern Logic, Find Differences) can be coded next using the same structure.
